Autonomous Orbital Maintenance: Diagnosis-Oriented Planning in PDDL and PDDL+

Michele Tozzola

1 Overview

This project models an autonomous orbital maintenance mission in which a robotic maintenance unit must diagnose and recover from faults occurring on an external spacecraft platform. Rather than treating planning as the pursuit of a single known goal state, the project centers on **diagnosis-oriented planning**: the robot does not initially know what is wrong with the system, and must actively gather information through diagnostic procedures before it can select an appropriate recovery action.

The work is split into two progressively more expressive planning models. **Q1** uses basic PDDL to model diagnosis, repair, and verification through discrete actions. **Q2** extends that model with PDDL+, adding continuous physical processes, events, and temporal behaviour. Although the two domains differ substantially in expressive power, they deliberately share the same logical reasoning architecture. Q2 is an extended version of Q1.

2 Problem and Domain Overview

The domain represents a robot performing maintenance on the exterior of an orbital platform. Over the course of a mission, components can develop faults such as stuck or leaking valves, faulty pressure sensors, or jammed inspection panels. None of these faults are directly observable, so the robot has to deduce the underlying cause from symptoms it can actually detect before attempting any repair.

Every maintenance task follows the same logical sequence:

$$Observe\_symptoms \rightarrow Run\_diagnostic\_tests \rightarrow Confirm\_the\_fault \rightarrow Repair \rightarrow Verify$$

Verification is mandatory after every repair. The robot never simply assumes a fix worked, it has to re-test the component and confirm it has actually returned to a healthy state.

3 Domain Objects

The model brings together several kinds of interacting objects.

The **physical infrastructure** of the platform is represented by tanks, valves, pressure sensors, inspection panels (introduced in Q2), toolboxes, and repair tools.

The reasoning process itself is driven by dedicated **diagnostic objects**, like faults, symptoms, diagnostic tests, and recovery procedures, rather than by logic hardcoded into the actions. These objects define the relationships between what can be observed and what might be causing it, and that relational data lives in the problem file rather than in the domain's operators.

The **robot** can move between locations, manipulate valves, collect tools, run diagnostic tests, repair components, and verify repairs. It has a single carrying hand, so it can never transport more than one tool at a time.

4 The Planning Problem

The objective isn't just to repair components, it's to produce a plan that identifies observable symptoms, discriminates between the possible faults those symptoms could indicate, selects the right recovery procedure, and verifies that the repair actually worked. This is what sets the problem apart from classical deterministic maintenance planning: some actions in the plan exist purely to gather information, not to change the state of the world toward the goal.

5 Diagnostic Logic

Diagnostic reasoning is approximated using explicit predicates and diagnostic actions. Symptoms give rise to a set of possible faults; once enough evidence has accumulated, exactly one of those faults becomes confirmed, and only a confirmed fault can be repaired. After repair, the component still has to pass a verification stage before it counts as operational again. In short, the workflow mirrors a typical industrial maintenance pipeline: observe, diagnose, confirm, repair, verify.

6 Fault Modelling

The project models several independent fault types, each with its own observable signature and recovery path.

A **stuck valve** is inferred when the valve is open but both of its neighbouring sensors report stable pressure — since pressure should normally shift across an open valve, stable readings on both sides strongly suggest a mechanical blockage. Recovery involves a mechanical procedure using the correctly sized wrench.

A **leaking valve** is inferred when the valve is closed but neighbouring sensors report changing pressure. Pressure shouldn't equalise across a closed valve, so any equalisation points to an unwanted leak. Recovery here is also mechanical.

A **sensor fault** is inferred when a pressure sensor either produces erratic readings or reports a stable pressure that contradicts the measurements of neighbouring sensors. Under the assumptions of this model, a sensor that observes a changing pressure is considered to be operating correctly. Because diagnosis is performed over a relatively short time horizon, gradual sensor drift is neglected, so a sensor capable of tracking pressure changes is assumed to be functioning normally.

Panel jams, introduced in Q2, occur during the opening operation of an inspection panel. Instead of moving as expected, the motor stalls, causing its current consumption to increase continuously until the alarm threshold is reached. Recovery consists of performing a lubrication procedure to gradually release the jam and restore normal operation.

The rules connecting observations to diagnoses can be summarised as follows:

None of these rules are hardcoded into the planner itself, they emerge from relationships defined in the problem file, which is central to how the project stays modular (more on that below).

Observation	Diagnosis
Sensor reports erratic reading	Sensor replacement required
Valve open, both sensors stable	Valve stuck
Valve open, one sensor changing, one stable	The stable sensor is faulty
Valve closed, both sensors changing	Valve leaking
Valve closed, changing pressure, unreliable sensor	Valve leaking

Table 1: Observation-to-diagnosis rules

7 Q1 — Classical PDDL Design

The first domain is built in basic PDDL, the entire reasoning process is carried by predicates.

The stuck-valve scenario is based on a real incident that occurred in 2010 on the International Space Station's (ISS). During the STS-131 mission, a critical valve on a newly installed ammonia coolant tank got stuck in the closed position, which compromised half of the International Space Station's thermal cooling capabilities. Because of the severity of the malfunction, NASA heavily considered adding an unplanned fourth spacewalk (EVA) and extending the shuttle's mission to manually fix the issue.

The world itself is represented as a graph of connected locations, with valves connecting tanks, sensors monitoring tanks, and the robot carrying a toolbox and repair tools. Movement happens across that location graph.

One of the most important design decisions was to cleanly separate **domain knowledge** from **world state**. Static knowledge, facts about how diagnosis works, such as which tests apply to which component, which symptoms a test requires, which faults a given result indicates, and which observations are considered unreliable, never changes during planning. Dynamic knowledge, observed symptoms, completed tests, possible and confirmed faults, repaired components, evolves as the plan unfolds. Keeping these apart is what makes the model as extensible as it is.

7.1 Diagnostic Actions

Rather than writing a custom action for every possible fault, the domain relies on a handful of **generic diagnostic patterns**: diagnosing a valve from two sensors, diagnosing from a single reliable sensor, analysing disagreement between sensors, and running a sensor self-test. Each of these actions contains almost no fault-specific logic, it simply interprets observations according to relationships already defined in the problem file.

7.2 Confirmation Phase

Once all required tests for a component have run, two generic actions decide the outcome. `confirm_fault` fires when every applicable test has completed and exactly one possible fault remains. This prevents repairing a fault that does not actually exist. `rule_out_fault` fires when every applicable test has completed and no fault hypothesis remains, marking the component healthy instead. Both actions rely on universal quantifiers, so they adapt automatically to any number of tests without needing to be rewritten.

7.3 Recovery

Repair is intentionally kept separate from diagnosis, and is handled by two generic mechanisms. **Mechanical recovery**, used for valve faults, requires a confirmed fault, a correctly sized wrench, and a matching recovery procedure. **Replacement recovery**, used for sensors, requires a spare sensor, safe working conditions, and the surrounding valves being closed.

7.4 Verification

Repair on its own is not sufficient; diagnostic tests must be repeated afterward. Only successful re-testing lets the component be marked operational. This mirrors how real maintenance procedures work, and it means the plan can never simply assume a repair succeeded.

8 Q2 — PDDL+ Design Choices

Q2 extends the classical model into PDDL+. The underlying logical reasoning barely changes; what's new is the ability to represent continuous physical behaviour, which brings in temporal planning, numeric fluents, processes, events, and continuous state evolution.

8.1 Continuous Processes

Unlike in Q1, the world in Q2 evolves on its own, independently of what the robot does. The robot has to plan around an environment that keeps changing.

Tank pressure equalisation is modelled as a continuous process: whenever a valve is open and the tanks it connects have different pressures, the process keeps updating tank mass and pressure until equilibrium is reached, approximating real fluid dynamics without the computational cost of modelling it exactly.

Panel opening is likewise continuous, a panel's position evolves over time through a movement process rather than flipping instantly between open and closed.

Lubrication recovery works the same way: once initiated, jam severity decreases gradually, and only once it reaches zero does an event declare the panel operational again.

8.2 Events

Events are changes that happen automatically once a condition becomes true and, unlike actions, they are triggered automatically rather than selected by the planner. Three events do the work here: pressure equilibrium stops fluid flow once the pressure difference between two tanks becomes small enough; alarm activation fires when the motor current of the manipulator exceeds a safety threshold, forcing the robot to respond; and jam cleared marks a panel operational once lubrication has reduced jam severity enough.

8.3 Numeric Fluents

Q2 introduces numeric variables in three groups: tank physics (pressure and mass, which evolve continuously during fluid transfer), motion (travelled distance, elapsed movement time, and movement duration, which together model realistic robot motion), and panel dynamics (panel position, jam severity, and motor current, which describe the inspection mechanism's physical behaviour).

8.4 How Actions, Processes, and Events Interact

The hybrid model splits responsibilities cleanly.

- **Actions**: `move`, `open_valve`, `repair_sensor`, `start_lubrication` ⇒ represent decisions the robot makes.
- **Processes**: `pressure_equalisation`, `panel_motion`, `lubrication_progress` ⇒ continuously evolve the world in the background.
- **Events**: `pressure_equalised`, `alarm_activated`, `jam_removed` ⇒ fire automatically once a threshold is crossed.

This mirrors how real hybrid control systems work: a controller issues commands while the physical plant it's controlling evolves continuously according to its own dynamics.

9 Why the Project Is Modular

Modularity was a central design goal throughout, and it shows up in a few concrete ways.

Diagnostic knowledge is data-driven. The diagnostic actions themselves are generic and contain no knowledge about individual faults, the actual relationships live in predicates like `applicable_test`, `test_requires_symptom`, `test_indicates`, and `unreliable_symptom`, all defined in the problem file. Adding a new fault, symptom, or diagnostic test is just a matter of adding facts. The planning operators never need to change.

Confirmation logic is generic. `confirm_fault` and `rule_out_fault` only check whether all required tests have completed and whether exactly one (or zero) hypotheses remain. Because that check is expressed with quantified predicates, adding more diagnostic tests never requires touching the confirmation logic.

Diagnosis, recovery, and verification are independent modules. Diagnosis produces `confirmed_fault`; recovery consumes it; verification confirms `component_ok`. Only a small number of predicates are shared across the three, and each stage has a single, well-defined responsibility.

Q2 extends Q1 without rewriting it. Perhaps the clearest demonstration of the architecture's modularity is that almost the entire diagnostic system carries over unchanged from Q1 to Q2. PDDL+ simply layers continuous flow physics, timed movement, panel dynamics, and numeric reasoning on top of it, with only panel diagnosis and lubrication recovery added as genuinely new pieces. The confirmation, verification, and repair framework is fully reused.

10 Mathematical Models and Continuous Dynamics

PDDL+ doesn't solve differential equations directly, so Q2 approximates real physical behaviour using simplified numeric relationships, chosen to keep the planning problem tractable while still capturing realistic qualitative behaviour.

Tank-to-tank mass flow is modeled as a continuous process whenever a connecting valve is open and a pressure gradient exists. The **mass flow rate** ($\dot{m}$) is calculated using the formula:

$$\dot{m} = k \cdot (P_{src} - P_{dest}) \cdot o$$

Here, P_{src} and P_{dest} are the source and destination pressures, k is the `flow_coefficient` (a PDDL fluent), and o is the `valve_opening` (another PDDL fluent). This is a refined pressure-driven

relationship that stands in for something like Bernoulli's equation, trading physical precision for a model that gets the qualitative behaviour right while making the influence of the valve state and distinct parameters explicit.

Pressure is not assumed to scale linearly with stored mass. Instead, the continuous rate of change of pressure ($\dot{P}$) for each tank is derived directly from the **ideal gas law relationship** ($PV = mRT$). Under assumptions of constant tank volume (V) and constant contents temperature (T), this simplifies to the differential equation

$$\dot{P} = \frac{RT}{V}\dot{m}$$

This derived relationship allows pressure to update realistically based on mass flow rate and applied per-tank. The **pressure equilibrium event** watches for $|P_{src} - P_{dest}| < \varepsilon$ and stops the flow process once the two pressures are close enough that no meaningful gradient remains.

Robot motion is timed rather than instantaneous: a movement's duration is $t = \frac{d}{v}$, where d is the path length and v is the robot's speed. A process increases elapsed travel time continuously until it reaches t, at which point a movement-completion event places the robot at its destination.

Panel opening follows a similar pattern: the `panel_position` fluent evolves as $\mathit{position} = \mathit{position} + \mathit{speed} \cdot \Delta t$ until the panel reaches fully open or fully closed. If the panel is jammed, though, it can't move — instead the **motor stall model** has current increase as $I = I + r \cdot \Delta t$, capturing how a stalled actuator behaves, and the **alarm activation** event fires once $I \geq I_{limit}$, introducing the panel-jam symptom the planner can then diagnose.

Lubrication recovery reverses that process: $\mathit{severity} = \mathit{severity} - r_{lub} \cdot \Delta t$, and once $\mathit{severity} \leq 0$ an event removes the fault and restores normal operation.

11 Q1 vs. Q2

Q1	**Q2**
Classical STRIPS planning	Hybrid PDDL+ planning
Instantaneous actions	Timed actions
Predicate reasoning only	Numeric reasoning
Static environment	Continuously evolving environment
Diagnosis only	Diagnosis + physical simulation
Discrete movement	Continuous movement
No autonomous evolution	Processes and events change the world independently

Table 2: Comparison between Q1 and Q2

12 Technical Discussion

12.1 Limitations of Classical PDDL

Classical PDDL assumes a fully observable world, making it unsuitable for realistic diagnosis. The predicates `possible_fault`, `confirmed_fault`, and `shows` simulate uncertainty, but the true fault is still explicitly encoded in the problem definition rather than hidden from the planner. Classical PDDL also lacks support for exogenous events, whereas PDDL+ allows faults and physical processes to evolve autonomously without planner intervention.

12.2 Diagnostic Actions vs. Goal-Achieving Actions

	Diagnostic actions	**Goal-achieving / repair actions**
Effect on physical world	None, they only add/remove conceptual predicates	Change actual component state
Purpose	Reduce uncertainty / narrow hypothesis space	Move the world toward the goal
Preconditions	Depend on symptoms already observed, not on the true fault	Gated by `confirmed_fault`
Reversibility	Effectively monotonic	Physically consequential and sometimes constrained

Table 3: Diagnostic actions vs. goal-achieving actions

12.3 Fault Progression and Planning Urgency

Fault progression introduces urgency because continuous processes can drive the system toward irreversible failure. For example, pressure equalisation caused by a stuck valve eventually triggers a failure event that ends the mission. Consequently, the planner must complete diagnosis, repair, and verification before these critical thresholds are reached, making time an essential planning resource.

12.4 Toward a POMDP-Based Model

A natural extension is to model diagnosis as a POMDP, replacing symbolic fault predicates with a probabilistic belief state over possible faults. Observations would become noisy rather than deterministic, continuous fault progression could be stochastic, and repair actions could have uncertain outcomes. Instead of producing a single plan, the planner would compute a policy that balances information gathering, repair costs, and the risk of failure under uncertainty.

13 Conclusion

This project shows how planning can be used not just to reach a goal, but to actively gather information and reason under uncertainty. The classical PDDL model provides a modular framework for diagnosis, confirmation, repair, and verification built entirely on symbolic reasoning, and the PDDL+ extension preserves that architecture while adding continuous physical behaviour, numeric state, and environmental evolution through processes and events. The result is a system that keeps logical reasoning cleanly separated from physical simulation. It is modular, extensible, and a reasonable step toward modelling increasingly realistic autonomous maintenance missions.